POO COM RUBY

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

Diferente de outras linguagens onde a Orientação a Objetos é apenas uma opção, no Ruby ela é a essência. Criada para ser "natural, não simples", Ruby foi desenhada para priorizar a produtividade e a “felicidade” do programador.
O que torna o Ruby único?

- POO Pura: Em Ruby, absolutamente tudo é um objeto – desde um número inteiro até uma string ou uma classe.
- Sintaxe Humana: O código é tão limpo e fluido que muitas vezes se lê como uma frase em inglês, reduzindo a carga cognitiva.
- Flexibilidade (Mixins): Através dos Módulos, o Ruby oferece uma forma elegante de compartilhar comportamentos entre classes sem as complicações da herança múltipla.

"Dominar Ruby é entender que a linguagem deve se adaptar ao seu raciocínio, e não o contrário. Vamos transformar lógica em elegância."

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

- 🔸 Composição → dependência total
- 🔹 Agregação → ligação fraca (referência)

CONSTRUTORES E COLECTIONS

1. **Arrays (Coleções Dinâmicas):**
   - O Poder do Array: No Ruby, o Array é a ferramenta principal. Esqueça nomes complexos como "List<T>" ou "Vetores"; o Array já é uma coleção dinâmica por padrão.
   - Tamanho Variável: Você pode adicionar ou remover objetos a qualquer momento sem se preocupar com o tamanho inicial da lista.
   - Polimorfismo Fluido: Graças à tipagem dinâmica, um único Array pode armazenar objetos de diferentes classes filhas de forma totalmente natural, facilitando o processamento de grupos.

INTERFACES

Interfaces em ruby são implementadas via módulos

- Ruby não tem interfaces formais.
- Modules definem contratos de comportamento.
- Classes podem “incluir” módulos com include.

```ruby
module Movimentavel
  def mover
    raise NotImplementedError, "Implementar em classe"
  end
end

class Robo
  include Movimentavel
  def mover; puts "Robo em movimento!"; end
end
```

EXERCÍCIO 1

Cenário: Em Cyberpunk 2077, o Afterlife é o bar mais famoso de Night City, uma metrópole futurista, e é frequentado por mercenários. O cardápio tem coquetéis comuns e drinks que homenageiam lendas. Todo drink tem um nome, que nunca muda, e um preço, que o bar pode reajustar dentro da tabela da casa.

Tarefa:

1. Crie a classe CoquetelAfterlife, cujo initialize recebe nome e preco:
   - nome pode ser lido de fora, mas não alterado (use attr_reader);
   - o preço fica guardado em @_preco, sem attr_reader, e o initialize atribui o valor recebido diretamente a ele.
2. Em CoquetelAfterlife, crie o getter manual preco e o setter manual preco=(valor). O setter só aceita valores de 5 a 200. Fora dessa faixa, ele exibe "Preço inválido!" e mantém o valor anterior.

EXERCÍCIO 1

3. Ainda em CoquetelAfterlife, crie o método ficha, que retorna (não imprime) um texto com o nome e o preço do drink.
4. Crie a classe filha DrinkDeLenda, com o atributo exclusivo lenda (com attr_reader):
    - o initialize recebe nome, preco e lenda e repassa os dois primeiros ao pai com super(nome, preco);
    - sobrescreva ficha, reaproveitando o texto do pai com super e acrescentando a lenda homenageada.
5. Na main, crie um array com um CoquetelAfterlife e dois DrinkDeLenda, à sua escolha, e percorra-o com .each, imprimindo a ficha de cada drink. Depois, tente definir, com o setter, o preço de um dos drinks de lenda como 500 e depois como 80, e imprima a ficha dele de novo.
6. Tente alterar o nome de um dos drinks diretamente no main (objeto.nome = ...) e observe o erro. Depois, transforme essa linha em comentário (#) para o programa voltar a rodar.

EXERCÍCIO 2

Cenário: Em Your Lie in April, jovens músicos de música clássica se apresentam em competições e concertos de gala. Em uma apresentação, cada instrumentista tem um papel: o solista toca a melodia principal e o acompanhador toca a base, seguindo o ritmo do solista. Todos são conduzidos pelo regente.

Tarefa:

1. Abstração: crie a classe base Instrumentista, cujo initialize recebe nome e instrumento (os dois com attr_reader). Crie também o método interpretar(peca), cujo corpo é apenas um raise NotImplementedError, com uma mensagem à sua escolha.
2. Crie a classe filha Solista, sem initialize próprio (ela usa o do pai), e sobrescreva interpretar(peca) para imprimir o nome, o instrumento e a peça, dizendo que ele conduz a melodia principal.

EXERCÍCIO 2

3. Crie a classe filha Acompanhador, também sem initialize próprio, e sobrescreva interpretar(peca) para imprimir o nome, o instrumento e a peça, dizendo que ele acompanha o ritmo do solista.
4. Crie a classe Regente, cujo initialize recebe nome e cria o array @orquestra vazio, com os métodos:
   - convocar(instrumentista), que adiciona o instrumentista ao @orquestra com <<;
   - reger(peca), que imprime o nome do regente e a peça e, com .each, chama interpretar(peca) de cada instrumentista;
   - escalacao, que usa .map para retornar um novo array de textos, um por instrumentista, com o nome da classe dele (use .class) e o seu nome.
5. Na main, crie um Solista, um Acompanhador e um Regente, à sua escolha. Convoque os dois instrumentistas, chame reger(...) com uma peça à sua escolha e imprima, com .each, cada texto do array retornado por escalacao.
6. Crie um Instrumentista diretamente e, na linha seguinte, chame interpretar(...) nele. Observe que o objeto é criado, mas a chamada gera um erro. Depois, transforme a linha da chamada em comentário (#) para o programa voltar a rodar.

EXERCÍCIO 3

Cenário: Em Steins;Gate, um grupo de amigos mantém o Laboratório de Gadgets do Futuro, um pequeno laboratório de invenções em Tóquio. A invenção mais importante do grupo é o PhoneWave, um micro-ondas ligado a um celular que, por acidente, envia mensagens de texto para o passado. O aparelho foi montado dentro do laboratório e só existe nele (composição). Os membros do laboratório já eram pessoas comuns antes de entrar no grupo e continuam existindo fora dele (agregação).

Tarefa:

1. Crie a classe PhoneWave, com o método enviar_dmail(mensagem), que imprime a mensagem e avisa que ela foi enviada para o passado.
2. Crie a classe MembroDoLab, cujo initialize recebe numero (o número de Lab Mem) e nome, os dois com attr_reader.

EXERCÍCIO 3

3. Crie a classe LaboratorioDeGadgets, cujo initialize não recebe parâmetros e:
    - cria um PhoneWave dentro dele, guardado em @phonewave;
    - cria o array @membros vazio.
4. Em LaboratorioDeGadgets, crie os métodos:
    - admitir(membro), que adiciona um membro já criado ao @membros e imprime o número e o nome dele;
    - experimento(mensagem), que imprime, com .each, o número e o nome de cada membro presente e depois envia a mensagem pelo enviar_dmail do PhoneWave guardado em @phonewave.
5. No main, crie dois membros antes do laboratório, à sua escolha. Depois, crie o laboratório, admita os dois e chame experimento(...) com uma mensagem à sua escolha.
6. Explique, em um comentário no código, o que é composição e o que é agregação no seu programa.

EXERCÍCIO 4

Cenário: Sherlock Holmes é um detetive que mora na Baker Street, em Londres. Ao investigar um crime, ele ouve todos os envolvidos, e cada um presta depoimento do seu jeito: a testemunha ocular conta o que viu e o suspeito apresenta o seu álibi. No fim, Holmes entrega à Scotland Yard quem pode ser detido, e só os suspeitos podem.

Tarefa:

1. Módulos: crie dois módulos:
   - Depoente (contrato), com o método depor, cujo corpo é apenas um raise NotImplementedError;
   - Detivel (implementação pronta), com o método ser_detido, que imprime o nome da pessoa e que ela foi levada pela Scotland Yard. O nome vem do attr_reader da classe que incluir o módulo.
2. Crie a classe base Envolvido, cujo initialize recebe nome (com attr_reader). Ela não inclui nenhum módulo.

EXERCÍCIO 4

3. Crie a classe TestemunhaOcular, que herda de Envolvido e inclui apenas Depoente. Implemente depor para imprimir o nome e o que a testemunha viu (texto à sua escolha).
4. Crie a classe SuspeitoDoCrime, que herda de Envolvido e inclui Depoente e Detivel. Implemente depor para imprimir o nome e o álibi do suspeito (texto à sua escolha).
5. Agregação: crie a classe Investigacao, cujo initialize recebe caso (o nome do caso) e cria o array @envolvidos vazio, com os métodos:
   - registrar(envolvido), que adiciona um envolvido já criado ao array;
   - ouvir_todos, que imprime o nome do caso e chama depor de cada envolvido com .each (polimorfismo);
   - acionar_scotland_yard, que usa .select com respond_to?(:ser_detido) para separar quem pode ser detido e chama ser_detido de cada um deles.
6. Na main, crie uma testemunha ocular e dois suspeitos, à sua escolha, antes da investigação. Depois, crie a investigação com um caso à sua escolha, registre os três e chame ouvir_todos e acionar_scotland_yard.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**



1 Contributor | 0 Issues | 1 Star | 0 Forks

zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

GitHub